

Bitcoin: un sistema de efectivo electrónico de igual a igual

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Resumen. Una versión puramente peer-to-peer del dinero electrónico permitiría el uso en línea los pagos se enviarán directamente de una parte a otra sin pasar por un institución financiera. Las firmas digitales proporcionan parte de la solución, pero la principal los beneficios se pierden si se sigue necesitando a un tercero de confianza para evitar el doble gasto. Proponemos una solución al problema del doble gasto mediante una red de igual a igual. La red marca la hora de las transacciones agrupándolas en una cadena continua de prueba de trabajo basada en hash, que forma un registro que no se puede cambiar sin volver a hacerlo la prueba de trabajo. La cadena más larga no solo sirve como prueba de la secuencia de eventos presenciados, pero prueba de que proviene de la mayor fuente de potencia de CPU. Como siempre y cuando la mayor parte de la potencia de la CPU esté controlada por nodos que no cooperen con si atacan la red, generarán la cadena más larga y superarán a los atacantes. El la red en sí misma requiere una estructura mínima. Los mensajes se transmiten con el máximo esfuerzo base, y los nodos pueden salir de la red y volver a unirse a ella a voluntad, aceptando el mayor tiempo cadena de prueba de trabajo como prueba de lo que ocurrió mientras no estaban.

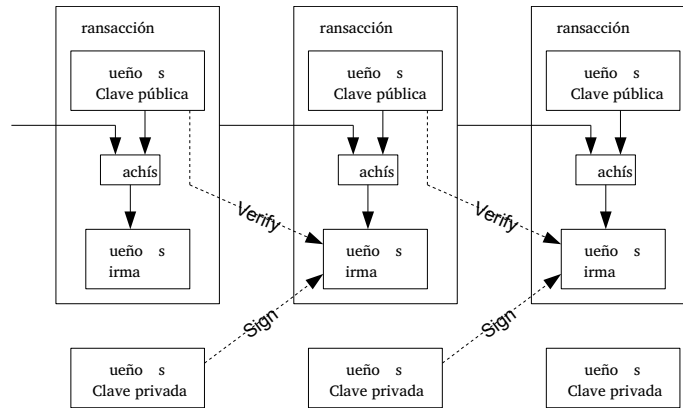
1. Introducción

El comercio en Internet ha pasado a depender casi exclusivamente de las instituciones financieras que actúan como terceros de confianza para procesar los pagos electrónicos. Si bien el sistema funciona lo suficientemente bien para en la mayoría de las transacciones, todavía adolece de las debilidades inherentes del modelo basado en la confianza. Las transacciones completamente irreversibles no son realmente posibles, ya que las instituciones financieras no pueden evite mediar en disputas. El costo de la mediación aumenta los costos de transacción, lo que limita la el tamaño mínimo práctico de las transacciones y la eliminación de la posibilidad de pequeñas transacciones ocasionales, y la pérdida de la capacidad de efectuar pagos irreversibles en caso de gastos no reversibles acarrea un costo mayor servicios reversibles. Con la posibilidad de una reversión, aumenta la necesidad de confianza. Los comerciantes deben desconfíe de sus clientes, molestándolos para obtener más información de la que necesitarían de otro modo. Se acepta un cierto porcentaje de fraude como inevitable. Estos costos e incertidumbres en los pagos se puede evitar en persona mediante el uso de moneda física, pero no existe ningún mecanismo para realizar pagos a través de un canal de comunicación sin una parte de confianza.

Lo que se necesita es un sistema de pago electrónico basado en pruebas criptográficas en lugar de en la confianza, permitir que dos partes que lo deseen realicen transacciones directamente entre sí sin la necesidad de una tercera parte. Las transacciones cuya anulación no sea práctico desde el punto de vista computacional protegerían a los vendedores contra el fraude, y los mecanismos rutinarios de depósito en garantía podrían implementarse fácilmente para proteger a los compradores. En En este artículo, proponemos una solución al problema del doble gasto utilizando un sistema distribuido de igual a igual servidor de marcas de tiempo para generar pruebas computacionales del orden cronológico de las transacciones. El el sistema es seguro siempre que los nodos honestos controlen colectivamente más potencia de CPU que cualquier otro grupo cooperante de nodos atacantes.

• transacciones

Definimos una moneda electrónica como una cadena de firmas digitales. Cada propietario transfiere la moneda al siguiente, firmando digitalmente un hash de la transacción anterior y la clave pública del siguiente propietario y añadiéndolos al final de la moneda. El beneficiario puede verificar las firmas para verificar la cadena de propiedad.

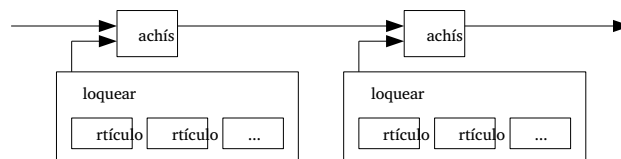


El problema, por supuesto, es que el beneficiario no puede verificar que uno de los propietarios no haya gastado dos veces la moneda. Una solución común es introducir una autoridad central de confianza, o casa de moneda, que compruebe cada transacción para duplicar el gasto. Después de cada transacción, la moneda debe devolverse a la Casa de la moneda para emitir una nueva moneda, y solo se confía en que las monedas emitidas directamente por la casa de la moneda no se gastarán dos veces. El problema con esta solución es que el destino de todo el sistema monetario depende de una empresa que gana la moneda, y cada transacción tiene que pasar por ella, al igual que un banco.

Necesitamos una forma de que el beneficiario sepa que los propietarios anteriores no firmaron ninguna antes de las transacciones. Para nuestros propósitos, la transacción más temprana es la que cuenta, por lo que no nos importa sobre los intentos posteriores de duplicar el gasto. La única forma de confirmar la ausencia de una transacción es estar al tanto de todas las transacciones. En el modelo basado en la ceca, la casa de la moneda estaba al tanto de todas las transacciones y decidió cuál llegó primero. Para lograr esto sin una parte de confianza, las transacciones deben ser anunciadas públicamente, y necesitamos un sistema para que los participantes se pongan de acuerdo sobre una historia única de la orden en la que se recibieron. El beneficiario necesita una prueba de que, en el momento de cada transacción, el la mayoría de los nodos estuvieron de acuerdo en que fue el primero en recibirse.

• servidor de marca de tiempo

La solución que proponemos comienza con un servidor de marcas de tiempo. Un servidor de marcas de tiempo funciona tomando un hash de un bloque de elementos para marcar la hora y publicar ampliamente el hash, como en un periódico o correo de Usenet. La marca de tiempo demuestra que los datos deben haber existido en tiempo, obviamente, para meterse en el libro. Cada marca de tiempo incluye la marca de tiempo anterior en su hash, que forma una cadena, y cada marca de tiempo adicional refuerza las anteriores.



No es necesario que las transmisiones de nuevas transacciones lleguen a todos los nodos. Mientras lleguen muchos nodos, entrarán en un bloque en poco tiempo. Las transmisiones en bloque también toleran la interrupción de mensajes. Si un nodo no recibe un bloque, lo solicitará cuando reciba el siguiente bloque y se da cuenta de que se perdió uno.

• Incentivo

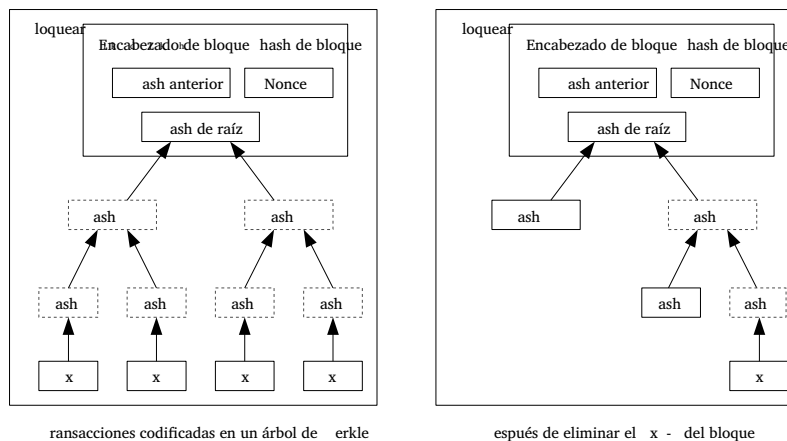
Por convención, la primera transacción de un bloque es una transacción especial que da inicio a una nueva moneda en propiedad del creador del bloque. Esto añade un incentivo para que los nodos apoyen la red y proporciona una forma de distribuir inicialmente las monedas en circulación, ya que no existe una autoridad central que las emita. La adición constante de una cantidad constante de monedas nuevas es análoga a lo que gastan los mineros de oro recursos para añadir oro a la circulación. En nuestro caso, lo que se gasta es el tiempo de CPU y la electricidad.

El incentivo también se puede financiar con comisiones de transacción. Si el valor de salida de una transacción es inferior a su valor de entrada, la diferencia es una comisión de transacción que se suma al valor de incentivo de el bloque que contiene la transacción. Una vez que haya ingresado un número predeterminado de monedas en circulación, el incentivo puede pasar por completo a comisiones de transacción y ser completamente ineficaz.

El incentivo puede ayudar a alentar a los nodos a mantenerse honestos. Si un atacante codicioso es capaz de ensamblar más potencia de CPU que todos los nodos honestos, tendría que elegir entre usarla para defraudar a la gente robándole sus pagos o utilizándolos para generar nuevas monedas. Debería hacerlo le resulta más rentable seguir las reglas, reglas que lo favorecen con más monedas nuevas que todos los demás juntos, no para socavar el sistema y la validez de su propia riqueza.

• Recuperar espacio en disco

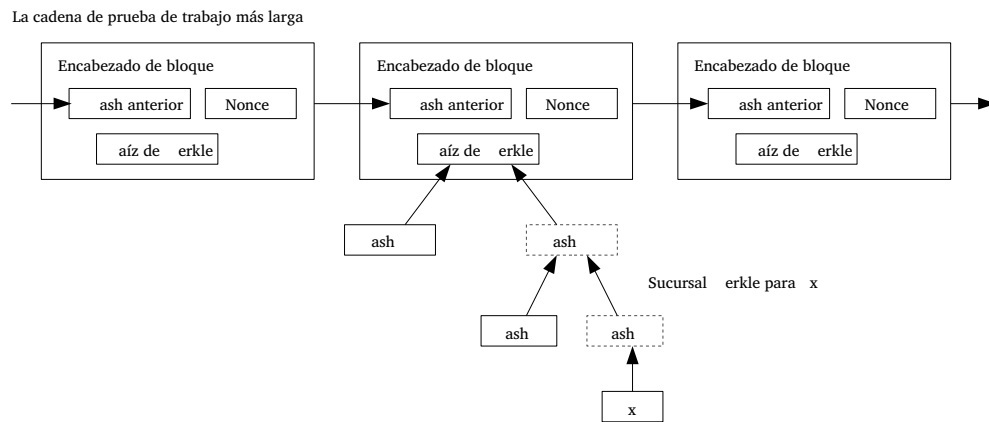
Una vez que la última transacción de una moneda está enterrada bajo suficientes bloques, las transacciones gastadas antes se puede desechar para ahorrar espacio en disco. Para facilitar esto sin romper el hash del bloque, las transacciones se cifran en un árbol de Merkle, y solo la raíz se incluye en el hash del bloque. Luego, los bloques viejos se pueden compactar quitando las ramas del árbol. Los hashes interiores sí no es necesario almacenarlo.



Un encabezado de bloque sin transacciones tendría unos 100 bytes. Si suponemos que los bloques son generados cada 10 minutos, 100 bytes por segundo, 86400 segundos por día, 365 días por año. Con sistemas informáticos normalmente se vendía con 100 MB en 1990, y la Ley de Moore predice el crecimiento actual de 100 MB al año, el almacenamiento no debería ser un problema, incluso si los encabezados de los bloques deben mantenerse en memoria.

• Verificación de pagos simplificada

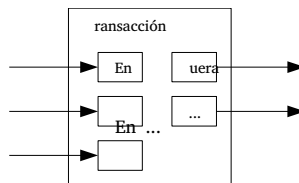
Es posible verificar los pagos sin ejecutar un nodo de red completo. Un usuario solo necesita mantener una copia de los encabezados de bloque de la cadena de prueba de trabajo más larga, que puede obtener consultando red nodos hasta que esté convencido de que tiene la cadena más larga y obtenga la sucursal de Merkle vincular la transacción al bloque en el que está marcada la hora. No puede comprobar la transacción él mismo, pero al vincularlo a un lugar de la cadena, puede ver que un nodo de la red lo ha aceptado, y los bloques agregados después de confirmar que la red lo ha aceptado.



Como tal, la verificación es confiable siempre que los nodos honestos controlen la red, pero es más vulnerable si la red está dominada por un atacante. Si bien los nodos de la red pueden verificar transacciones por sí mismas, el método simplificado puede ser engañado por un atacante fabricado transacciones mientras el atacante pueda seguir dominando la red. Una estrategia para protegerse contra esto sería aceptar alertas de los nodos de la red cuando detecten un error bloquear, lo que hacía que el software del usuario descargara el bloque completo y alertaba a las transacciones a confirme la incoherencia. Es probable que las empresas que reciben pagos frecuentes sigan queriendo hacerlo ejecutan sus propios nodos para una seguridad más independiente y una verificación más rápida.

• Combinar y dividir el valor

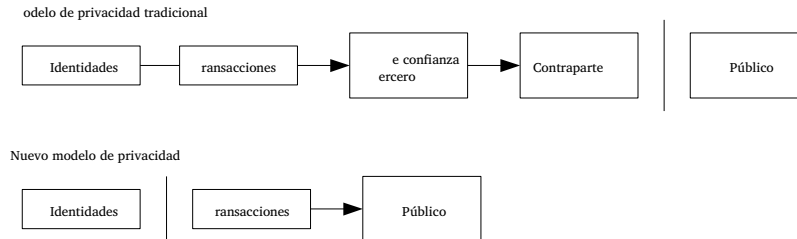
Aunque sería posible manipular las monedas de forma individual, sería difícil hacer una transacción separada por cada centavo de una transferencia. Para permitir que el valor se divida y combine, las transacciones contienen múltiples entradas y salidas. Normalmente habrá una sola entrada de una transacción anterior más grande o de múltiples entradas que combinan cantidades más pequeñas y, como máximo, dos salidas: una para el pago y otra para devolver el cambio, si lo hubiera, al remitente.



Cabe señalar que la distribución, en la que una transacción depende de varias transacciones, y aquellas las transacciones dependen de muchas más, no es un problema aquí. Nunca existe la necesidad de extraer una copia completa e independiente del historial de una transacción.

1 . Privacidad

El modelo bancario tradicional logra un nivel de privacidad al limitar el acceso a la información al las partes involucradas y el tercero de confianza. La necesidad de anunciar públicamente todas las transacciones excluye este método, pero la privacidad aún se puede mantener interrumpiendo el flujo de información en otro lugar manteniendo las claves públicas anónimas. El público puede ver que alguien está enviando una cantidad para otra persona, pero sin información que vincule la transacción con nadie. Esto es similar al nivel de información publicado por las bolsas de valores, donde el tiempo y el tamaño de las operaciones individuales, la cinta, se hacen públicas, pero sin decir quiénes eran las partes.



Como firewall adicional, se debe usar un nuevo par de claves para cada transacción a fin de mantenerlas de estar vinculado a un propietario común. Algunos enlaces siguen siendo inevitables con entradas múltiples transacciones, que necesariamente revelan que sus insumos pertenecían al mismo propietario. El riesgo es que si se revela el propietario de una clave, la vinculación podría revelar otras transacciones que pertenecieron a el mismo propietario.

11. Ataques

Consideramos el escenario de un atacante que intenta generar una cadena alternativa más rápido que lo normal. Incluso si esto se logra, no deja al sistema expuesto a cambios arbitrarios, como crear valor de la nada o llevarse dinero que nunca perteneció al agresor. Los nodos no aceptará una transacción inválida como pago, y los nodos honestos nunca aceptarán un bloque que los contiene. Un atacante solo puede intentar cambiar una de sus propias transacciones para recuperarla dinero que gastó recientemente.

La carrera entre la cadena honesta y la cadena del atacante puede caracterizarse como un binomio Caminata aleatoria. El éxito es que la cadena honesta se extiende una cuadra, aumentando su liderados por 1 , y el evento de error es que la cadena del atacante se extiende un bloque, lo que reduce la diferencia de -1 .

La probabilidad de que un atacante recupere un déficit determinado es análoga a la de un jugador Problema de ruina. Supongamos que un jugador con crédito ilimitado comienza con un déficit y juega potencialmente un número infinito de intentos para tratar de alcanzar el punto de equilibrio. Podemos calcular la probabilidad de que alguna vez alcance el punto de equilibrio o que un atacante alcance alguna vez la cadena honesta, de la siguiente manera

- p probabilidad de que un nodo honesto encuentre el siguiente bloque
- q probabilidad de que el atacante encuentre el siguiente bloque
- q_z probabilidad de que el atacante alguna vez alcance a una cuadra por detrás

$$q_z = \begin{cases} p & \text{si } p > q \\ q & \text{si } p \leq q \end{cases}$$

ado nuestro supuesto de que $p < q$, la probabilidad cae exponencialmente a medida que el número de bloques el atacante tiene que ponerse al día con los aumentos. Con las probabilidades en su contra, si no tiene suerte Si se lanza hacia adelante desde el principio, sus posibilidades se vuelven cada vez más pequeñas a medida que se queda más atrás.

hora consideramos cuánto tiempo debe esperar el destinatario de una nueva transacción antes de ser suficientemente seguro de que el remitente no puede cambiar la transacción. Suponemos que el remitente es un atacante quien quiere hacer creer al destinatario que le pagó durante un tiempo y luego cambiarlo para devolverlo a él mismo después de que haya pasado algún tiempo. El receptor recibirá una alerta cuando eso suceda, pero el remitente espera que sea demasiado tarde.

El receptor genera un nuevo par de claves y entrega la clave pública al remitente poco antes firma. Esto evita que el remitente prepare una cadena de bloques con antelación trabajando en de forma continua hasta que tenga la suerte de avanzar lo suficiente, y luego ejecuta la transacción en ese momento. Una vez que se envía la transacción, el remitente deshonesto comienza a trabajar en secreto en un cadena paralela que contiene una versión alternativa de su transacción.

El destinatario espera hasta que la transacción se haya agregado a un bloque y z los bloques han sido enlazado después de él. No sabe la cantidad exacta de progreso que ha logrado el atacante, pero asumiendo que los bloques honestos duraron el tiempo promedio esperado por bloque, el potencial del atacante el progreso será una distribución de Poisson con un valor esperado

$$\frac{z \cdot q}{p}$$

Para obtener la probabilidad de que el atacante aún pueda alcanzarlo ahora, multiplicamos la densidad de Poisson por cada cantidad de progreso que podría haber logrado según la probabilidad de que pudiera ponerse al día desde ese punto

$$\frac{e^{-\lambda} \lambda^z}{z!} \cdot \frac{q^z p^{1-z}}{1 - \frac{q}{p}}$$

reorganizando para evitar sumar la cola infinita de la distribución...

$$\frac{e^{-\lambda} \lambda^z}{z!} \cdot \frac{q^z p^{1-z}}{1 - \frac{q}{p}}$$

Convirtiendo a código C...

```

include
double probabilidad_de_éxito_del_atacante (double q, int z)
{
    double p = 1 - q;
    double lambda = z * q / p;
    suma = 0;
    for (int i = 0; i <= z; i++)
    {
        double veneno = exp(-lambda);
        suma += veneno * (lambda^i / i!) * (q^i * p^(z-i));
    }
    return suma;
}

```

Al analizar algunos resultados, podemos ver que la probabilidad cae exponencialmente con z .

[illegible][illegible]

resolviendo una P inferior al , ...

P ,
q . Z
q . Z
q , Z
q , Z
q , Z
q , Z
q . Z
q , Z

1 . conclusión

emos propuesto un sistema para las transacciones electrónicas sin depender de la confianza. Empezamos con el marco habitual de las monedas hechas a partir de firmas digitales, que proporciona un fuerte control de propiedad, pero está incompleta sin una forma de evitar el doble gasto. Para resolver esto, nosotros propuso una red de igual a igual que utilizara pruebas de trabajo para registrar un historial público de transacciones que rápidamente se vuelve poco práctico desde el punto de vista computacional para que un atacante cambie si son nodos honestos controlan la mayoría de la potencia de la CPU. La red es sólida en su simplicidad desestructurada. Nodos trabajan de una sola vez con poca coordinación. No es necesario identificarlos, ya que los mensajes son no se envían a ningún lugar en particular y solo deben entregarse haciendo todo lo posible. Los nodos pueden salir y volver a unirse a la red a su antojo, aceptando la cadena de prueba de trabajo como prueba de lo que ocurrió mientras no estaban. otan con la potencia de su CPU, expresando su aceptación de bloques válidos trabajando para ampliarlos y rechazar bloques no válidos negándose a trabajar en ellos. odas las reglas e incentivos necesarios se pueden hacer cumplir con este mecanismo de consenso.

Referencias

1. Wei Dai, b-money, <http://www.weidai.com/bmoney.txt>, 1997.
2. David S. Johnson, S. J. Kilian y J. K. S. Liu, Diseño de un servicio seguro de marca de tiempo con un mínimo de requisitos de confianza, en *Journal of Cryptology*, vol. 10, núm. 1, mayo de 1997, páginas 39-55.
3. S. Haber, S. Stornetta, Cómo sellar el tiempo en un documento digital, en *Journal of Cryptology*, vol. 10, núm. 1, mayo de 1997, páginas 57-71.
4. S. Haber, S. Stornetta, Mejorar la eficiencia y la confiabilidad de la marca de tiempo digital, en *Journal of Cryptology*, vol. 10, núm. 1, mayo de 1997, páginas 73-84.
5. S. Haber, S. Stornetta, Nombres seguros para cadenas de bits, en *Journal of Cryptology*, vol. 10, núm. 1, mayo de 1997, páginas 85-96.
6. S. Haber, S. Stornetta, La vuelta de hashcash: una contramedida de denegación de servicio, en *Journal of Cryptology*, vol. 10, núm. 1, mayo de 1997, páginas 97-108.
7. C. P. W. Kerle, Protocolos para criptosistemas de clave pública, en *Journal of Cryptology*, vol. 10, núm. 1, mayo de 1997, páginas 109-120.
8. S. J. Kilian, Introducción a la teoría de la probabilidad y sus aplicaciones, 1997.